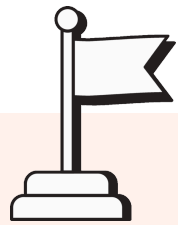


Student Academic Performance and Risk Prediction System



Project Overview



Objective: To propose, design, and develop a complete machine learning pipeline for predicting student academic outcomes.

Target Outcomes:- Graduate- Dropout- Enrolled

1. Real-World Impact

- Student dropout affects millions of students worldwide
- Early prediction can save students' academic careers

2. Social Relevance

- Education is fundamental to societal development
- Reducing dropout rates improves workforce quality

3. Technical Challenge

- Multi-class classification problem
- Imbalanced dataset requires careful evaluation
- Opportunity to compare multiple algorithms



4. Complete ML Pipeline Experience

- From data preprocessing to deployment
- Covers all course learning outcomes

Problem Statement

The Problem:

Student dropouts from higher education result in:

- Losses to universities financially
- Unskilled workforce
- Limited progress academically by the students

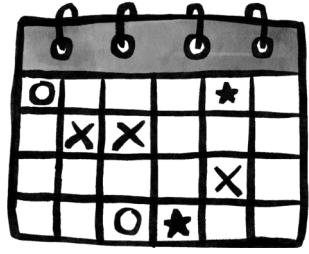
Present Problem:

Present means to determine students at risk are REACTIVE, and are only employed once the problems have grown serious.

Our Solution:

Using machine learning techniques in a PROACTIVE system that makes early predictions of student success/failure.

Methodology Summary



Step 1: Data Acquisition



Step 2: Data Cleaning



Step 3: Data Exploration



Step 4: Splitting Data into Training and Testing Sets



Step 5: Data Normalization



Step 6: Model Training (4 Models)



Step 7: Model Evaluation



Step 8: Model Selection & Saving



Step 9: Streamlit Deployment



Features Summary (36 Features)

Personal Details:

- Marital status, Gender, Age at enrolment, International Student

Educational Details:

- Application mode, Application sequence, Course Code
- Previous qualification and mark, Admission Mark
- Daytime/Evening Attendance

Family Details:

- Mother's and Father's qualification and occupation

Financial Details:

- Debtor, Tuition fees, Scholarship Holder

Educational Achievement:

- First Semester: credited, enrolled, evaluated, approved, mark
- Second Semester: credited, enrolled, evaluated, approved, mark

Economic Indicators:

- Unemployment, Inflation, GDP



Preprocessing Steps

1. Data Loading

- CSV file loaded with semicolon separator using pandas library

2. Missing Values Detection

- No missing values detected

3. Removing Duplicates

- No duplicates detected

4. Target Variable Mapping

- Dropout - 0
- Enrolled - 1
- Graduate - 2

5. Train and Test Sets Splitting

- 80% for training (3,539 records)
- 20% for testing (885 records)



```
[3] ✓ 0s df["Target"].value_counts()
```

...	count
Target	
Graduate	2209
Dropout	1421
Enrolled	794

dtype: int64

Metrics for Evaluation

Given that this is a problem of multiclass classification, the following metrics were utilized to analyze our model:

Accuracy

- Total number of correct predictions / total predictions

Precision (weighted)

- The number of selected items that are relevant

Recall (weighted)

- The number of relevant items that have been selected

F1-Score (weighted)

- The harmonic mean between precision and recall

Confusion Matrix

- Shows actual vs predicted for each class

```
results_df = pd.DataFrame(results)
results_df = results_df.sort_values(by="F1-Score", ascending=False)
```

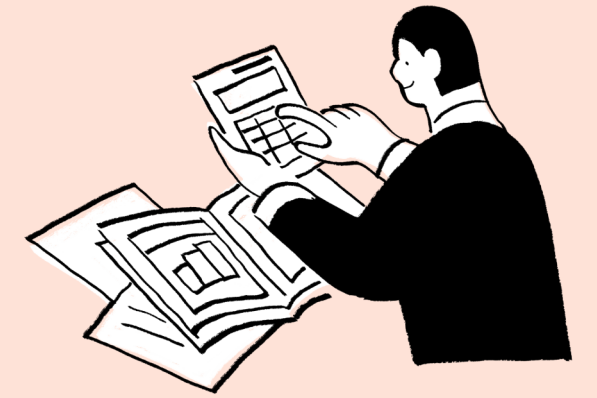
	Model	Accuracy	Precision	Recall	F1-Score
2	Random Forest	0.767232	0.754445	0.767232	0.753903
0	Logistic Regression	0.768362	0.749981	0.768362	0.753128
1	Decision Tree	0.697175	0.701903	0.697175	0.699416
3	KNN	0.667797	0.648311	0.667797	0.655282

Next steps:

[Generate code with results_df](#)

[New interactive sheet](#)

Models Used (4 Models)



Model 1: Logistic Regression

- Simple linear model
- Intuitive and speedy
- Good for classification tasks

Model 2: Decision Tree

- Nonlinear model
- Simple to understand visually
- Handles different types of data

Model 3: Random Forest

- Ensemble technique (ensemble of decision trees)
- Decreases overfitting
- Captures more interactions

Model 4: K-Nearest Neighbors (KNN)

- Example-based model
- Ideal for small data sets
- Depends on distance functions

```
▶ models = {  
    "Logistic Regression": LogisticRegression(max_iter=1000),  
    "Decision Tree": DecisionTreeClassifier(random_state=42),  
    "Random Forest": RandomForestClassifier(random_state=42),  
    "KNN": KNeighborsClassifier()  
}  
  
results = []  
trained_models = {}
```